

IMAGO Media Finder

Architecture Overview

IMAGO Media Finder is a lightweight Next.js application built with TypeScript and Tailwind CSS. The application provides a searchable interface for inconsistent media metadata from seed.json.

The system is split into three main layers:

Search Core

Implemented in `src/lib/search.ts`. This layer loads the seed dataset, preprocesses metadata, builds an in-memory search index, applies relevance scoring, filters, sorting, and pagination.

API Layer

Implemented with Next.js route handlers:

GET `/api/search`: returns paginated search results

GET `/api/facets`: returns available credits, restrictions, and date boundaries

GET `/api/analytics`: returns basic in-memory usage metrics

Frontend UI

Implemented in `src/app/page.tsx`. The UI includes a debounced keyword search input, credit filter, restriction multi-select, date range filters, date sorting, result cards, pagination, and loading/empty/error states.

The implementation avoids external services so the project remains easy to run, review and reason about.

Search Strategy And Relevance

Search is implemented using a lightweight in-memory index. Each media item is preprocessed into weighted searchable tokens.

The searchable fields are:

suchtext: primary content field

fotografen: secondary credit field

bildnummer: optional identifier field

derived restrictions from suchtext

The scoring model uses field weights:

suchtext: weight 6

fotografen: weight 3

bildnummer: weight 2

restrictions: weight 1

Exact token matches receive a stronger score than prefix matches. This makes direct matches more relevant while still supporting partial/prefix-style searches.

Default result ordering is:

Relevance score

Newest date

If the user selects date sorting, the API sorts by normalized date ascending or descending.

This approach was chosen because it is transparent, maintainable, and sufficient for the 10,000-item challenge dataset without introducing Elasticsearch, OpenSearch, or Meilisearch infrastructure.

Preprocessing Strategy

Preprocessing happens at server startup/module load time when `src/lib/search.ts` imports `seed.json` and builds the in-memory index.

Implemented preprocessing steps:

-Date Normalization

Converts datum from DD.MM.YYYY into ISO format:

30.03.1952 -> 1952-03-30

This enables reliable date range filtering and date sorting.

-Restriction Extraction

Extracts publication restriction tokens from `suchtext`, for example:

PUBLICATIONxINxGERxONLY

PUBLICATIONxNOTxINxJPN

These are normalized into readable filter values:

PUBLICATION IN GER ONLY

PUBLICATION NOT IN JPN

-Text Normalization

Text is normalized by lowercasing, removing diacritics, and splitting punctuation-like delimiters.

This helps searches match inconsistent metadata such as accented names, punctuation, or mixed casing.

-Tokenization And Stop-Word Handling

Text is split into searchable tokens. Common low-value words such as `the`, `and`, `und`, `der`, and `in` are removed so they do not distort relevance.

-Weighted Token Map

Each item receives a precomputed token map with field-specific weights. This avoids re-tokenizing every field on every request.

This preprocessing currently happens at runtime startup rather than as a persisted build artifact. For the challenge scale, this is a practical trade-off. In production, preprocessing should move into a build or ingestion pipeline.

Scaling Approach For Millions Of Items And Continuous Ingestion

The current in-memory scan is appropriate for 10,000 items but would not be the right production architecture for millions of records.

For a production-scale system, I would move to a dedicated search platform such as OpenSearch, Elasticsearch, or Meilisearch.

The production architecture would include:

- An ingestion pipeline that validates and normalizes incoming media metadata
- Structured fields for date, credit, restrictions, people, locations, countries, rights, and identifiers
- Full-text indexes with weighted fields
- Language-aware analyzers for multilingual captions
- Faceted indexes for fast filtering
- Autocomplete indexes for names, agencies, locations, teams, and image numbers
- Typo tolerance and phrase/proximity matching
- Incremental ingestion through a queue or change-data-capture process
- Versioned documents and reindexing with aliases for zero-downtime schema changes
- Observability for query latency, zero-result queries, popular keywords, and ranking quality

For continuous ingestion, new or updated media items would flow through a queue. A worker would normalize the metadata, extract structured fields, and update the search index. This keeps search serving separate from ingestion work and allows the index to scale independently.

For organic discoverability, I would also add server-rendered/crawlable pages for high-value search categories, canonical URLs for filter pages, structured metadata, and stable result URLs.

Testing Approach

The project includes focused tests for the search core in `src/lib/search.test.ts`.

Covered areas:

- Date parsing from DD.MM.YYYY to ISO format
- Restriction extraction from suchtext
- Keyword search matching
- Credit, date, and restriction filtering
- Date sorting

These tests focus on the highest-risk logic: preprocessing and search behavior.

I also verified the application with:

npm run lint
npm run test
npm run build

Recommended next testing steps:

Add Playwright tests for the main UI flows
Test keyboard navigation for filters and pagination
Add tests for empty and error states
Add performance benchmarks for larger generated datasets
Add API contract tests for edge cases such as invalid dates, empty queries, and large page sizes

Trade-Offs

The main trade-off was choosing a lightweight in-memory search index instead of an external search engine.

Benefits of the chosen approach:

- Easy to run locally
- No infrastructure dependency
- Clear and reviewable TypeScript implementation
- Good enough for the required 10,000-item dataset
- Preprocessing and scoring decisions are visible in code

Limitations:

- It scans the indexed array at request time
- It is not typo-tolerant
- It does not support advanced phrase/proximity ranking
- Analytics are stored in memory and reset on server restart
- The index is rebuilt on server startup
- It is not suitable for millions of items without replacing the search backend

Other trade-offs:

- Snippets are plain text rather than HTML-highlighted to avoid unsafe markup.
- The UI does not show media thumbnails because the seed data does not include image URLs.
- Restriction extraction is intentionally scoped to the PUBLICATIONx... pattern.
- The search model prioritizes clarity and maintainability over ranking sophistication.

Summary

This solution demonstrates a maintainable search layer for inconsistent metadata using preprocessing, weighted relevance scoring, filters, sorting, pagination, and a polished Tailwind UI.

The current implementation is intentionally lightweight and well-suited to the challenge dataset. For production scale, the same concepts would move into a dedicated search engine and ingestion pipeline.